

Modul *Programmieren I*

Projektdokumentation

Parkhaus-Simulation in C

Dokumenttyp: Projektdokumentation
Thema: Entwurf und Umsetzung einer Parkhaus-Simulation
Dozent: Prof. Dr. Christian Braunagel
Teamname: Three-Knights-at-Parking
Studienjahrgang: TSA/TSL25
Sudenten: Simon Ibach (6696863)
Luca Perri (2746139)
Daniel Weimer (5649354)
Datum: 18. März 2026

Dokumentation zu Konzeption, Umsetzung und Teamarbeit

Inhaltsverzeichnis

1	Projektrahmen	1
2	Umsetzung – Parkhaus-Simulation	1
2.1	Grundidee der Umsetzung	2
2.2	Zentrale Steuerung über Settings	2
2.3	Tickbasierter Simulationsablauf	3
2.4	Demand-Generierung und Gate-Routing	3
2.5	Gates, Queue und Einfahrlogik	4
2.6	General-Funktionen und Subticks	5
2.7	Fahrzeugmodell und GenericVehicle	5
2.8	Queue vor Parkhaus statt direkter Soforteinfahrt	5
2.9	Parkhaus, Belegung und Ausfahrt	6
2.10	Statistik pro Tick und Gesamtsummary	6
2.11	Speicherung, Laden und Ausgabe	7
2.12	Storage-Modul	8
2.13	Benutzeroberfläche als zustandsbasierte Struktur	8
2.14	Schnittstellen zwischen UI, Simulation und Datahandling	9
2.15	Robustheit und Fehlerbehandlung	9
3	Diskutierte Optionen	9
3.1	Monolithische UI vs. modulare UI	10
3.2	Direkte Konsolenausgabe / UI-Aufrufe vs. SaveHandler-Middleware	10
3.3	scanf() vs. kontrolliertes zeilenbasiertes Parsing	10
3.4	Starre Arrays vs. dynamische bzw. verkettete Strukturen	11
3.5	Direkte Implementierung spezifischer Fahrzeugtypen vs. abstrakte Zwischenebene GenericVehicle	11
3.6	Direkte Sofort-Summary vs. vollständige Speicherung jedes Ticks	11
3.7	Demand direkt pro Gate vs. zentrale Demand-Berechnung mit anschließendem Routing	12

3.8	Immer aktive Subticks vs. bedarfsabhängige Subticks	12
3.9	Globaler Zustand (Shared Memory) vs. kontrollierte Statuskommunikation und Error-Handling	12
3.10	Erweiterungen außerhalb des Pflichtumfangs	13
4	Entscheidungsgrundlagen zur Projektumsetzung	13
4.1	Modularität und klare Zuständigkeiten	14
4.2	Erweiterbarkeit	14
4.3	Robustheit und Wartbarkeit	15
4.4	Testbarkeit und geringere Verschachtelung	15
4.5	Nachvollziehbarkeit der Ergebnisse	15
4.6	Fairness und Realismus bei mehreren Gates	15
5	Organisation und Aufteilung im Team	15
5.1	Grundsätzliche Aufteilung	16
5.2	Zuständigkeitsverteilung im Team	16
5.3	Warum diese Aufteilung für uns sinnvoll war	17
5.4	Arbeitsorganisation im Verlauf des Projekts	17
5.5	Workflow und Abstimmung	18
6	Rückblick auf die Teamarbeit & Projekt	18
6.1	Herausforderungen in der Zusammenarbeit	19
6.2	Technische Herausforderungen	19
6.3	Was im Team gut funktioniert hat	20
6.4	Was uns leicht gefallen ist und was eher nicht	20
7	Fazit	20

1 Projektrahmen

Im Rahmen des Moduls *Programmieren I* bestand unsere Aufgabe darin, eine Parkhaus-Simulation in C zu entwerfen und umzusetzen. Die Grundlage dafür bildeten die Anforderungen aus Teil I und Teil II der Aufgabenstellung. Neben der eigentlichen technischen Umsetzung war für uns auch relevant, die Lösung so zu strukturieren, dass sie nachvollziehbar dokumentiert, im Team bearbeitet und schrittweise weiterentwickelt werden konnte.

Unser Projekt war dabei nicht nur eine reine Programmieraufgabe, sondern zugleich ein gemeinsamer Entwicklungsprozess. Wir mussten fachliche Anforderungen aus der Aufgabenstellung in eine konkrete Softwarestruktur überführen, unsere Arbeitsaufteilung im Team organisieren und den Umgang mit der Toolchain weiter erlernen, um eine sinnvolle Zusammenarbeit, Dokumentation und Kommunikation zu ermöglichen. Gerade zu Beginn war dieser Rahmen für das Projekt wesentlich, weil nicht nur die Simulationslogik selbst, sondern auch die gemeinsame Arbeitsweise sowie die thematische Aufteilung gefestigt werden mussten.

Die vorliegende Dokumentation schreiben wir bewusst aus unserer Perspektive als Team. Grundlage der Dokumentation ist die resultierende Simulationslogik aus Teil II, basierend auf der Planung und den konzeptionellen Überlegungen aus Teil I. Des Weiteren werden Entscheidungen aus Besprechungen, größtenteils festgehalten in Protokollen, sowie aus Kanban-Diskussionen und sonstigen Absprachen möglichst nachvollziehbar und ganzheitlich dargestellt.

Für diese Dokumentation war uns deshalb wichtig, den Projektrahmen ehrlich und nachvollziehbar zu beschreiben: nicht als idealisierten Ablauf, sondern als tatsächliches Teamprojekt mit klaren Anforderungen, technischer Struktur, organisatorischem Lernprozess und einer schrittweise konkretisierten Umsetzung.

2 Umsetzung – Parkhaus-Simulation

2.1 Grundidee der Umsetzung

Unser grundlegender Anspruch bei der Umsetzung war, den Code möglichst modular, klar trennbar und später weiterbearbeitbar aufzubauen. Wir wollten keine Lösung, in der alle Zuständigkeiten in wenigen großen Funktionen vermischt sind, sondern eine Struktur, bei der sich Konfiguration, Simulationslogik, Parkhauslogik, Statistik, Dateiverwaltung und Benutzerinteraktion sauber voneinander abgrenzen.

Daraus hat sich eine Aufteilung in mehrere Kernbereiche ergeben:

- ein zentrales `Settings`-Objekt für alle relevanten Simulationsparameter,
- ein `Simulation`-Objekt als Rahmen für den gesamten Ablauf,
- das `Parkhaus` als eigentliche Belegungs- und Bewegungslogik,
- pro Einfahrt (Gate) eine eigene Queue,
- eine allgemeine Zwischenebene `GenericVehicle`,
- aufbauend auf `GenericVehicle` konkrete Fahrzeugtypen,
- ein Statistiksysteem mit Tick-Daten und Gesamtsummary,
- eine Speicher- und Lade-Komponente,
- eine terminalbasierte Benutzeroberfläche mit getrennten UI-Modulen.

Damit war für uns früh klar, dass wir nicht einfach nur eine einzelne Simulationsschleife schreiben wollten, sondern ein System, das in Teilbereichen verständlich bleibt und sich später erweitern lässt.

2.2 Zentrale Steuerung über Settings

Eine grundlegende Designentscheidung war, die gesamte Simulation über ein zentrales `Settings`- bzw. Konfigurationsobjekt zu steuern. Alle wesentlichen Parameter der Simulation liegen damit an einer Stelle. Dazu gehören unter anderem:

- Name des Parkhauses,
- Kapazität und Anzahl der Stockwerke,
- Anzahl der Gates,
- Gate-Entry-Zeit pro Fahrzeug,
- Tick-Dauer in Sekunden,
- maximale Tick-Anzahl,

- maximale Queue-Länge,
- minimale und maximale Parkdauer,
- Eintrittswahrscheinlichkeit,
- Zufalls-Seed,
- Output-Modus.

Diese Bündelung war für uns wichtig, weil wir dadurch die Simulation konsistent initialisieren, validieren und an mehreren Stellen mit denselben Regeln arbeiten konnten. Gleichzeitig ließ sich die Benutzeroberfläche daran gut anbinden, weil nicht jede Komponente ihre eigenen Konfigurationswege pflegen musste.

2.3 Tickbasierter Simulationsablauf

Die Simulation selbst ist diskret tickbasiert aufgebaut. Das bedeutet, dass jeder Simulationsschritt als eigener Tick behandelt wird. Innerhalb eines Ticks gibt es eine feste Reihenfolge, weil wir das Verhalten deterministisch und nachvollziehbar halten wollten.

Der grobe Ablauf ist wie folgt:

1. Zu Beginn des Ticks wird der neue Statistik-Eintrag für diesen Tick vorbereitet.
2. Danach wird der Demand für diesen Tick zentral berechnet.
3. Dieser Demand wird anschließend auf die einzelnen Gate-Queues verteilt.
4. Danach wird im Parkhaus überprüft, ob Fahrzeuge in diesem Tick ausfahren.
5. Erst im Anschluss können neue Fahrzeuge in das Parkhaus einfahren.
6. Währenddessen werden die Tick-Statistiken fortgeschrieben.
7. Nach Abschluss des Ticks können die Daten ausgegeben und gespeichert werden.

Für uns war dabei besonders wichtig, dass das Parkhaus innerhalb eines Ticks immer zuerst entleert und erst danach befüllt wird. So entstehen keine unklaren Zustände, bei denen Fahrzeuge gleichzeitig noch als parkend und bereits als ausgefahren behandelt würden.

2.4 Demand-Generierung und Gate-Routing

Bei der Demand-Generierung wollten wir bewusst keine sekundenweise Mikrosimulation, sondern eine kontrollierbare und robuste Erzeugung pro Tick. Die zentrale Idee war deshalb, den gesamten Demand zuerst für den Tick als Ganzes zu berechnen und ihn erst danach auf die einzelnen Gates zu verteilen.

Diese Trennung hatte für uns mehrere Vorteile. Zum einen bleibt die Berechnung des Gesamtdemands an einer Stelle gebündelt. Zum anderen konnten wir dadurch die spätere Verteilung separat betrachten und offenhalten, etwa für Gewichtung einzelner Gates, unterschiedliche Zuflussrichtungen oder weitere Erweiterungen wie Stoßzeiten und tageszeitabhängige Veränderungen im Demand sowie in der Verteilung auf die einzelnen Einfahrten.

Im finalen Stand haben wir den Demand über eine Poisson-basierte Lösung mit Backup über eine Normalverteilung umgesetzt. Entscheidend war für uns dabei nicht nur ein realistischer Ansatz zur Demand-Generierung selbst, sondern vor allem die saubere Trennung zwischen:

- **Gesamtdemand berechnen** und
- **Demand auf Gates verteilen.**

In der zweiten Phase, also beim Gate-Routing, wird der zuvor berechnete Gesamtdemand auf die vorhandenen Gates verteilt. Wenn der Demand nicht exakt aufgeht, kann verbleibender Rest auf einzelne Gates verteilt werden. Diese Struktur erschien uns zentral, kontrollierbar und später gut erweiterbar.

2.5 Gates, Queue und Einfahrlogik

Die Anzahl der Gates ist frei definierbar. Jedes Gate entspricht einer Einfahrt und besitzt eine eigene Queue. Diese Queue funktioniert nach dem First-in-first-out-Prinzip. Fahrzeuge, die zuerst ankommen, werden also auch zuerst weiterverarbeitet. Zusätzlich ist jede Queue begrenzt. Damit wollten wir nicht nur das reale Verhalten vor dem Parkhaus abbilden, sondern auch einen klaren Punkt definieren, an dem Fahrzeuge abgewiesen werden müssen, weil sich nicht beliebig viele Fahrzeuge anstellen können.

Für die Einfahrt war uns wichtig, dass sie nicht unbegrenzt modelliert wird. Anders als die Ausfahrt, die wir vereinfacht als ungehindert angenommen haben, ist die Einfahrt durch eine Gate-Entry-Zeit pro Fahrzeug begrenzt. Dahinter steht die Annahme, dass der Fahrer eines Fahrzeugs Zeit benötigt, um an die Schranke zu fahren, eine Karte zu ziehen oder eine Erkennung abzuwarten und anschließend einzufahren. Für die Ausfahrt wurde angenommen, dass moderne Technologien zur Kennzeichenerkennung eingesetzt werden und dadurch eine nahezu ungehinderte Ausfahrt möglich ist. Dadurch entsteht pro Tick eine obere Grenze der möglichen Einfahrten.

Diese Grenze ergibt sich aus:

$$\text{maximale Einfahrten pro Tick} = \frac{\text{Tickzeit in Sekunden}}{\text{Gate-Entry-Zeit in Sekunden}}$$

Damit keine halben oder gebrochenen Einfahrten entstehen, wurde folgende Regel festgelegt, dass `tick_inSec` nur ein Vielfaches von `gate_entry_inSec` sein darf.

Diese Entscheidung hat den Code deutlich vereinfacht und robuster gemacht, weil wir kein zusätzliches Gate-Zwischenobjekt für Teil-Einfahrten einführen mussten.

2.6 General-Funktionen und Subticks

Strukturell haben wir allgemeine Grundfunktionen für das Entleeren und Befüllen des Parkhauses vorgesehen. Diese bilden die Basisfunktionalität ab. Zusätzlich gibt es eine Subtick-Logik, die nur dann relevant wird, wenn überhaupt mehr als ein Gate vorhanden ist.

Der Hintergrund war folgender: Wenn mehrere Gates existieren, wollten wir vermeiden, dass immer dieselbe Reihenfolge einzelne Gates systematisch bevorzugt. Durch eine Subtick-Behandlung konnten wir die Gates innerhalb eines Ticks paralleler betrachten und damit realistischer abarbeiten. Wenn hingegen nur ein Gate vorhanden ist, bringt diese zusätzliche Schicht keinen echten Mehrwert und würde die Ausführung nur unnötig komplizierter machen.

Deshalb haben wir uns gegen eine immer aktive Subtick-Simulation entschieden und stattdessen eine bedarfsabhängige Zusatzroutine verwendet.

2.7 Fahrzeugmodell und GenericVehicle

Ein weiterer wichtiger Bestandteil unserer Struktur war das `GenericVehicle`. Die Idee dahinter war, ein allgemeines Basisobjekt bereitzustellen, das den Großteil bzw. alle gemeinsamen Eigenschaften enthält. Darauf aufbauend können spezifische Fahrzeugtypen, wie aktuell das Auto, erweitert werden.

Für uns hatte das zwei Vorteile:

- Wir konnten Queue-, Listen- und Parkhauslogik mit einer gemeinsamen Zwischenebene umsetzen.
- Gleichzeitig haben wir die Struktur offen gehalten, um später weitere Fahrzeugtypen ergänzen zu können.

Das war für uns besonders wichtig, weil wir nicht wollten, dass eine mögliche spätere Erweiterung an einem zu eng gebauten Fahrzeugmodell scheitert. Auch wenn im aktuellen Stand die Simulation zunächst auf Autos fokussiert ist, ist die Struktur bereits so vorbereitet, dass spätere weitere Fahrzeugtypen relativ einfach ergänzt werden können.

2.8 Queue vor Parkhaus statt direkter Soforteinfahrt

Jedes Fahrzeug wird bei uns zunächst im Zusammenhang mit dem Füllprozess betrachtet. Das heißt, ein Fahrzeug muss zuerst in eine Queue gelangen und wird erst von dort aus ins Parkhaus übernommen. Wir haben uns bewusst gegen eine

direkte Soforteinfahrt entschieden.

Der Grund dafür war vor allem statistischer Natur: Wenn ein Fahrzeug bereits bei Ankunft erzeugt wird und zunächst in der Queue existiert, können wir sauber unterscheiden zwischen:

- dem Zeitpunkt der Ankunft am Parkhaus und
- dem Zeitpunkt der tatsächlichen Einfahrt.

Dadurch konnten wir Wartezeiten nachvollziehbar erfassen und die Queue nicht nur als technische Zwischenliste, sondern als echten Teil des Simulationsgeschehens behandeln.

2.9 Parkhaus, Belegung und Ausfahrt

Das Parkhaus selbst verwaltet die aktuell parkenden Fahrzeuge in einer dynamischen Struktur. Fahrzeuge werden während der Simulation gespeichert, durchlaufen den Parkvorgang und werden wieder entfernt, sobald ihr Leaving-Tick erreicht ist.

Bei der Einfahrt wird festgelegt, wie viele Stellplätze ein Fahrzeug belegt. Dabei haben wir auch die Möglichkeit schlechten Parkens berücksichtigt. Ein normal parkendes Fahrzeug belegt einen Stellplatz, ein schlecht parkendes Fahrzeug zwei. Diese Entscheidung wird bei der Einfahrt getroffen und wirkt sich dann direkt auf die Belegung des Parkhauses aus.

Beim Entleeren des Parkhauses wird die Liste parkender Fahrzeuge pro Tick durchlaufen. Wenn ein Fahrzeug seinen Leaving-Tick erreicht hat, wird es entfernt und die Belegung wird entsprechend reduziert. Da wir für die Ausfahrt keine separate Exit-Zeit modelliert haben, konnten wir diesen Teil vereinfachen und den Fokus auf die komplexere Einfahrlogik legen.

2.10 Statistik pro Tick und Gesamtsummary

Die Statistik haben wir nicht nur als Nebenprodukt gesehen, sondern als einen zentralen Teil der Aufgabenlösung. Ursprünglich lag der Gedanke nahe, pro Tick einige Werte zu erzeugen und diese relativ direkt in eine Gesamtsummary einfließen zu lassen. Im Verlauf des Projekts haben wir uns aber bewusst dafür entschieden, jeden Tick einzeln zu speichern.

Dafür wird pro Tick ein eigenes Statistikobjekt erzeugt und an eine Statistikliste angehängt. Erst am Ende der Simulation wird daraus eine zusammenfassende `StatsSummary` berechnet.

Diese Entscheidung hatte für uns mehrere Vorteile:

- vollständige Nachvollziehbarkeit jedes Ticks,

- saubere Trennung zwischen Laufzeitdaten und Endauswertung,
- gute Grundlage für spätere Diagramme, Grafiken oder weitere Analysen,
- keine irreversiblen Informationsverluste durch zu frühe Verdichtung.

Damit konnten wir nicht nur die Mindestanforderung von mindestens fünf Statistiken pro Zeitschritt erfüllen, sondern zugleich eine deutlich bessere Datengrundlage für spätere Auswertungen schaffen.

2.11 Speicherung, Laden und Ausgabe

Neben der Simulation selbst haben wir auch die Speicherung und das spätere Laden von Statistikdaten berücksichtigt. Die Tick-Daten und Summaries können gespeichert werden. Zusätzlich wurde eine Logik vorgesehen, um diese Daten wieder einzulesen und über die UI darzustellen.

Bei den Ausgaben war uns wichtig, dass der Normal-Modus die Anforderungen zuverlässig erfüllt. Erweiterte Modi wie Verbose oder Debug verstehen wir als zusätzliche Ebenen, nicht als Ersatz für die Pflichtausgabe. Deshalb haben wir die Ausgabe nicht nur technisch, sondern auch als strukturelle Frage behandelt: Was muss immer sichtbar sein, und was ist nur für ausführlichere Analyse sinnvoll?

Eine zentrale Rolle übernimmt dabei der `SaveHandler` als Middleware zwischen Simulationslogik, Dateiverwaltung und Benutzeroberfläche. Zu Beginn der Entwurfsphase hatten wir diskutiert, ob die Simulation ihre Ergebnisse nach jedem Tick direkt per `printf()` ausgeben oder direkte UI-Funktionsaufrufe auslösen sollte. Das wäre zunächst die einfachste und schnellste Lösung gewesen. Wir haben diesen Ansatz jedoch bewusst verworfen, weil dadurch Berechnungslogik und Darstellungslogik untrennbar miteinander verzahnt worden wären.

Mit dem `SaveHandler` haben wir stattdessen eine eigenständige Schicht eingeführt. Die Simulation übergibt ihre Datenpakete an diese Middleware, welche sich anschließend um Persistierung, Dateimanagement und die Weitergabe an die UI kümmert. Dadurch kann die Kernlogik der Simulation unabhängig von Ausgabemodus, Dateiformat oder konkreter Darstellung weiterentwickelt werden.

Nach Übergabe der geladenen Simulationsdaten in Form einer `StatList` spielt die Benutzeroberfläche eine zentrale Rolle bei deren Interpretation und Darstellung. Die vom `SaveHandler` bereitgestellten Tick-Daten (der `StatList` angehängt) werden nicht direkt ausgegeben, sondern zunächst durch die UI strukturiert aufbereitet.

Die Ausgabe erfolgt dabei bewusst tickbasiert, sodass der zeitliche Verlauf der Simulation nachvollziehbar bleibt. Für jeden Tick werden relevante Kennzahlen entsprechend der Ausgabe-Modi dargestellt. Diese Werte werden im Terminal in einer kompakten, konsistenten Form ausgegeben. Am Ende werden die formatierten `StatsSummary`-Werte ausgegeben, um dem Benutzer die abschließende Simulationsübersicht zu bieten.

Ein weiterer Aspekt war die bewusste Trennung zwischen Simulationslogik und Dar-

stellung. Die Simulation erzeugt ausschließlich Daten, während die Benutzeroberfläche deren Aufbereitung und Ausgabe übernimmt. Dadurch bleibt die Benutzeroberfläche unabhängig vom Simulationsvorgang und kann flexibel gestaltet, erweitert oder angepasst werden.

2.12 Storage-Modul

Im Bereich des Storage-Moduls wurde anfangs ein deutlich umfangreicherer Ansatz diskutiert. Dabei war vorgesehen, dem Benutzer eine Navigation durch eine Verzeichnisstruktur zu ermöglichen, um Simulationen in Ordnern zu organisieren, auszuwählen und gegebenenfalls zu verwalten.

Die zugrunde liegende Idee bestand darin, eine Struktur aufzubauen, in der Simulationsergebnisse nach Programmlaufzeiten gruppiert und über mehrere Ebenen zugänglich gemacht werden. Für eine solche Lösung wären jedoch zusätzliche Funktionen zur Verzeichnisnavigation, Dateiverwaltung sowie zur internen Abbildung dieser Strukturen notwendig gewesen.

Im Verlauf der Umsetzung haben wir diesen Ansatz bewusst verworfen. Ausschlaggebend dafür war insbesondere, dass ein solches System über die eigentlichen Anforderungen der Aufgabenstellung hinausgeht und ein eigenständiges Modul zur Dateiverwaltung erfordert hätte. Der zusätzliche Implementierungsaufwand hätte den Fokus von der Kernaufgabe der Simulation und ihrer Auswertung abgelenkt.

Stattdessen haben wir eine vereinfachte und zielgerichtete Lösung umgesetzt. Diese ermöglicht es, entweder die zuletzt erzeugten Simulationsergebnisse zu laden oder eine konkrete Datei über einen benutzerdefinierten Pfad einzulesen. Auf diese Weise bleibt der Zugriff auf gespeicherte Daten gewährleistet, ohne zusätzliche Komplexität in die Systemarchitektur einzuführen.

2.13 Benutzeroberfläche als zustandsbasierte Struktur

Die Benutzeroberfläche wurde als zustandsbasierte Struktur umgesetzt. Dabei wird jeder Menübereich durch einen klar definierten Zustand repräsentiert. Die Navigation erfolgt über eine zentrale Steuerschleife, in der abhängig vom aktuellen Zustand das entsprechende Menü aufgerufen und der nächste Zustand zurückgegeben wird.

Die möglichen Zustände umfassen unter anderem das Hauptmenü, die Konfiguration, die Simulation, das Laden gespeicherter Daten (Storage) sowie Hilfeseiten. Durch diese Struktur bleibt der Programmfluss klar nachvollziehbar und lässt sich bei Bedarf um weitere Menüpunkte erweitern.

Strukturell wurde die Benutzeroberfläche in mehrere spezialisierte Module aufgeteilt. Dazu gehören insbesondere Module für das Hauptmenü, die Konfiguration, die Simulationssteuerung, die Statistikdarstellung, das Laden von Simulationsdaten sowie Hilfetexte. Diese Aufteilung folgt dem allgemeinen Ansatz des Projekts, Zuständigkeiten klar zu trennen und einzelne Komponenten unabhängig weiterentwickeln

zu können.

2.14 Schnittstellen zwischen UI, Simulation und Datahandling

Ein zentraler Aspekt der Architektur ist die klar definierte Schnittstelle zwischen Benutzeroberfläche, Simulationslogik und Datenverwaltung.

Die Interaktion zwischen UI und Simulation erfolgt primär über Steuerfunktionen. Insbesondere wird die Simulation durch einen expliziten Funktionsaufruf aus der Benutzeroberfläche gestartet. Dabei werden die aktuell gesetzten Settings (im Simulations-Objekt hinterlegt) an die Simulation übergeben, welche anschließend eigenständig den gesamten Simulationsablauf steuert.

Während der Laufzeit der Simulation erfolgt die Ausgabe der Daten nicht durch die UI selbst, sondern wird aktiv durch das Simulationsbackend angestoßen. Hierfür stellt die Benutzeroberfläche entsprechende Ausgabefunktionen zur Verfügung, beispielsweise zur Darstellung einzelner Tick-Daten sowie der abschließenden Gesamtauswertung.

Das Backend ruft diese Funktionen gezielt zu den jeweiligen Zeitpunkten im Simulationsablauf auf. Dadurch bleibt die zeitliche Kontrolle vollständig innerhalb der Simulation, während die UI ausschließlich für die Formatierung und Ausgabe verantwortlich ist.

Ein alternativer Ansatz, bei dem die UI den Simulationsverlauf kontinuierlich überwacht und eigenständig entscheidet, wann neue Daten ausgegeben werden, wurde bewusst nicht verfolgt. Eine solche Lösung hätte eine deutlich stärkere Kopplung zwischen UI und Simulation erfordert und zusätzliche Synchronisationslogik notwendig gemacht. Durch den gewählten Ansatz wird stattdessen eine klare Rollenverteilung erreicht: Die Simulation steuert den Ablauf und bestimmt die Zeitpunkte der Ausgabe, während die UI die Darstellung übernimmt.

Beim Laden gespeicherter Simulationen erfolgt die Datenbereitstellung hingegen über das Datenverwaltungsmodul. In diesem Fall werden die eingelesenen Datenstrukturen an die UI, zur Ausgabe im Terminal, übergeben (vgl. Abschnitt 2.11).

2.15 Robustheit und Fehlerbehandlung

Bei der Implementierung haben wir bewusst darauf geachtet, Zwischenschritte möglichst früh zu prüfen. Unerwartete Zustände sollten nicht stillschweigend übergangen, sondern sichtbar gemacht werden. Dahinter stand für uns kein Selbstzweck, sondern die Erfahrung, dass gerade in einem modular aufgebauten C-Projekt viele Probleme nur dann sauber beherrschbar bleiben, wenn Fehler früh und lokal erkannt werden.

Dieser Ansatz war auch für die Zusammenarbeit wichtig, weil sich Probleme dadurch klarer einer Komponente oder Schnittstelle zuordnen ließen.

3 Diskutierte Optionen

Im Verlauf des Projekts haben wir mehrere Struktur- und Umsetzungsvarianten diskutiert. Nicht jede davon war von Anfang an ausgeschlossen; viele Entscheidungen haben sich erst im Laufe der Entwurfs- und Implementationsphase klar herausgebildet.

3.1 Monolithische UI vs. modulare UI

Eine naheliegende Möglichkeit wäre gewesen, die gesamte Benutzeroberfläche in einer einzelnen Datei abzubilden. Das hätte den Einstieg zunächst einfacher gemacht. Wir haben uns aber dagegen entschieden, weil wir schnell gesehen haben, dass eine monolithische UI bei Menüs, Konfiguration, Simulationssteuerung, Statistikansicht, Storage und Hilfetexten schnell unübersichtlich wird.

Deshalb haben wir die UI in mehrere Module aufgeteilt, unter anderem für Home, Konfiguration, Simulation, Statistik, Storage und Hilfe.

3.2 Direkte Konsolenausgabe / UI-Aufrufe vs. SaveHandler-Middleware

Zu Beginn der Entwurfsphase diskutierten wir, ob die Simulationslogik ihre Ergebnisse nach jedem Tick direkt über `printf()` ausgeben oder direkte Funktionsaufrufe an die Benutzeroberfläche tätigen sollte. Dies wäre kurzfristig die mit Abstand schnellste und einfachste Implementierung gewesen.

Wir haben diesen Ansatz jedoch bewusst verworfen, da er die Berechnungslogik untrennbar mit der Darstellungslogik verzahnt hätte. Jede spätere Änderung, etwa das Hinzufügen eines CSV-Exports, die Anpassung des Konsolen-Layouts oder die Einführung unterschiedlicher Output-Modi, hätte tiefe Eingriffe in die Kernlogik der Simulation erfordert.

Die Entscheidung fiel daher auf die Einführung des `SaveHandlers` als Middleware. Dieser Ansatz bedeutet zwar initial einen höheren Architekturaufwand, isoliert das I/O-Handling jedoch vollständig vom Zustand der Simulation.

3.3 `scanf()` vs. kontrolliertes zeilenbasiertes Parsing

Für die Benutzereingaben hätten wir direkt mit `scanf()` arbeiten können. Diese Variante haben wir jedoch bewusst nicht bevorzugt.

Stattdessen wurde ein kontrolliertes, zeilenbasiertes Einlesen implementiert. Dabei wird zunächst die gesamte Eingabe mittels `fgets()` gelesen und anschließend mit `strtol()` bzw. `strtod()` in numerische Werte umgewandelt.

Dieses Vorgehen bietet mehrere Vorteile. Zum einen können ungültige Eingaben,

wie etwa nicht-numerische Zeichen oder zusätzliche Eingabeteile, zuverlässig erkannt werden. Zudem wurde eine Validierungsschicht eingeführt, die sicherstellt, dass eingegebene Werte innerhalb definierter Grenzen liegen. Fehlerhafte Eingaben bringen somit kein undefiniertes Verhalten hervor, sondern werden erkannt und führen zu einer erneuten Eingabeaufforderung.

Auch für die Verarbeitung von Texteingaben wird derselbe zeilenbasierte Ansatz verwendet. Eingaben werden zunächst vollständig eingelesen und anschließend geprüft bzw. weiterverarbeitet. Dadurch bleibt das Verhalten konsistent, unabhängig davon, ob numerische oder textuelle Daten verarbeitet werden.

Im Vergleich zu `scanf()` erhöht dieser Ansatz die Robustheit der Anwendung deutlich und reduziert typische Probleme wie Buffer-Reste oder unkontrollierte Eingabeverarbeitung.

3.4 Starre Arrays vs. dynamische bzw. verkettete Strukturen

Schon während der Entwurfsphase wurde früh klar, dass es keinen Sinn macht, alle Daten entweder ausschließlich mit starren Arrays oder ausschließlich mit dynamischen Listen zu organisieren. Listen wie die einzelnen Queues der Eingänge wurden als Array konfiguriert, hingegen die Fahrzeuge in der Queue selbst sowie die parkenden Fahrzeuge als verkettete Strukturen. So wurde je nach Einzelfall die sinnvollste Option gewählt, um Flexibilität, Datensicherheit und eine möglichst effiziente Umsetzung zu vereinen.

3.5 Direkte Implementierung spezifischer Fahrzeugtypen vs. abstrakte Zwischenebene `GenericVehicle`

Wir hätten auch direkt nur einen konkreten Fahrzeugtyp verwenden können. Für den ersten und aktuellen Stand der Anforderungen wäre dies zunächst ausreichend gewesen. Wir haben uns dennoch für eine allgemeine Zwischenebene entschieden, weil wir die spätere Erweiterbarkeit von Anfang an mitberücksichtigen wollten.

Würde man anstelle des `GenericVehicle` nur ein spezifisches `Car` verwenden, wären sämtliche Listen, Queues und die Kernlogik des Parkhauses fest an diesen einen Typen gebunden. Spätere Erweiterungen um andere Fahrzeugtypen wären dann deutlich aufwendiger.

3.6 Direkte Sofort-Summary vs. vollständige Speicherung jedes Ticks

Eine weitere Option war, Tick-Daten nur flüchtig zu berechnen und sofort in eine Gesamtsummary zu überführen sowie lediglich wenige Statistiken in einer Datei oder der CUI-Ausgabe festzuhalten. Das hätte Speicher und Struktur vereinfacht. Wir haben uns dagegen entschieden, weil wir die Tick-Ebene vollständig erhalten

wollten, um genaue Analysen im Anschluss an die Simulation zu ermöglichen.

Gerade für Debugging, die Analyse von Stau-Situationen vor den Gates und mögliche spätere grafische Auswertungen war es für uns sinnvoller, die vollständige Historie beizubehalten.

3.7 Demand direkt pro Gate vs. zentrale Demand-Berechnung mit anschließendem Routing

Auch bei der Demand-Erzeugung gab es mehrere Umsetzungsmöglichkeiten. Eine Möglichkeit wäre gewesen, für jedes Gate den Demand separat zu erzeugen. So wäre jedoch eine prozentuale Wahrscheinlichkeit für ein neues Fahrzeug für jede einzelne Einfahrt nötig gewesen. Des Weiteren wäre die Generierung des neuen Demands unübersichtlicher geworden.

Wir haben uns stattdessen für eine saubere Trennung entschieden. Zuerst wird der Demand erzeugt und anschließend auf die einzelnen Gates verteilt. Zentral im Mittelpunkt stand bei dieser Entscheidung die Möglichkeit, relativ einfach auch einen zeit- bzw. tagesabhängigen Demand zu generieren.

3.8 Immer aktive Subticks vs. bedarfsabhängige Subticks

Ein weiterer Diskussionspunkt war, ob jede Simulation grundsätzlich auf Subtick-Ebene arbeitet oder ob diese zusätzliche Komplexität nur bei mehreren Gates nötig ist. Wir haben uns gegen eine generelle Subtick-Pflicht entschieden und die Subtick-Routine nur für den Mehr-Gate-Fall vorgesehen. Diese Entscheidung wurde zudem stark durch den Ansatz bestimmt, die Simulation möglichst recheneffizient zu gestalten.

3.9 Globaler Zustand (Shared Memory) vs. kontrollierte Statuskommunikation und Error-Handling

In der Entwurfsphase stand außerdem die Frage im Raum, wie Zustände und insbesondere Fehler zwischen den verschiedenen Modulen kommuniziert werden. Eine mögliche Variante wäre ein globaler Shared-Memory-Ansatz gewesen, bei dem globale Variablen genutzt werden, in die beliebige Funktionen unkontrolliert Statusupdates oder Fehlermeldungen schreiben.

Diesen Ansatz haben wir bewusst nicht weiterverfolgt. Stattdessen wollten wir Statuskommunikation klar definieren, damit Ownership und Verantwortlichkeit erhalten bleiben. Aus diesem Grund wurden feste Return-Codes und klar nachvollziehbare Fehlerpfade vorgesehen.

3.10 Erweiterungen außerhalb des Pflichtumfangs

Zusätzlich wurden auch Ideen wie Diagramme, weitergehende Exportformate oder sogar mehrere parallele Simulationen kurz angesprochen. Diese Themen waren interessant, standen für uns aber nicht im Zentrum der Kernabgabe. Deshalb haben wir sie nicht als tragende Basis der finalen Projektstruktur gewählt, jedoch die Möglichkeit einer späteren Integration solcher Funktionen berücksichtigt.

4 Entscheidungsgrundlagen zur Projektumsetzung

Unsere Entscheidungen zur Projektstruktur und zur konkreten Umsetzung sind nicht zufällig gefallen, sondern orientierten sich an mehreren wiederkehrenden Grundsätzen. Dabei haben wir sowohl Inhalte aus der Vorlesung als auch bereits vorhandene praktische Erfahrungen aus privaten Programmier- und Projektkontexten einbezogen. Für uns war wichtig, Entscheidungen nicht vorschnell zu treffen, sondern verschiedene Ansätze zunächst zu prüfen, im Team zu besprechen und erst dann final festzulegen, wenn die zugrunde liegende Sachlage ausreichend nachvollzogen werden konnte. Wir haben also versucht, strukturelle und technische Aussagen möglichst fundiert zu bewerten und uns jeweils für die Variante zu entscheiden, die innerhalb des Projekts sinnvoll, nachvollziehbar und mit einem erkennbaren Mehrwert verbunden war.

4.1 Modularität und klare Zuständigkeiten

Der wichtigste architektonische Grundsatz war für uns die klare Trennung von Verantwortlichkeiten im Sinne des Separation-of-Concerns-Prinzips. Indem wir Berechnungslogik, Datenhaltung, Middleware und Darstellungslogik sauber getrennt haben, entstand eine modulare und nachvollziehbare Softwarearchitektur.

Wenn Settings, Simulation, Parkhaus, Queue, Statistik, SaveHandler und UI jeweils eine erkennbare Rolle haben, bleibt der Code verständlicher und Änderungen lassen sich gezielter vornehmen. So kann beispielsweise ein UI-Layout angepasst oder ein neues Dateiformat integriert werden, ohne die Kernlogik der Simulation verändern zu müssen.

4.2 Erweiterbarkeit

Viele Entscheidungen haben wir bewusst so getroffen, dass spätere Erweiterungen möglich bleiben. Das gilt insbesondere für:

- das `GenericVehicle` als Basis für weitere Fahrzeugtypen,
- die Trennung von Demand-Erzeugung und Gate-Routing,
- die modulare Simulation,
- die vollständige Tick-Speicherung,
- die getrennten Output-Modi,
- die modulare UI-Struktur.

Wir wollten vermeiden, dass die Architektur zwar kurzfristig funktioniert, aber bei späteren Anpassungen an zu eng gefassten Grundannahmen scheitert.

4.3 Robustheit und Wartbarkeit

Ein weiterer zentraler Maßstab war die Systemstabilität. Da C keine automatische Speicherbereinigung bietet und unsauberer Code schnell zu undefiniertem Verhalten führt, haben wir uns für eine defensive Programmierung entschieden. Deshalb haben wir uns zum Beispiel gegen `scanf()` entschieden, halbe Einfahrten über die Vielfach-Regel ausgeschlossen und die Zuständigkeiten bei Status- und Speicher-verwaltung bewusst enger gefasst.

Gerade in C war uns wichtig, dass Ownership, Speicherfreigabe und Listenmanipulationen nicht implizit bleiben. Das hat zwar an manchen Stellen mehr Abstimmung erfordert, war für die Stabilität und Wartbarkeit des Gesamtprojekts aber notwendig.

4.4 Testbarkeit und geringere Verschachtelung

Komplexe, stark verschachtelte Funktionen sind in C schwer zu warten und fehleranfällig. Daher haben wir großen Wert darauf gelegt, Teilaufgaben klar zu isolieren. Die Trennung zwischen Entleeren des Parkhauses und anschließendem Befüllen war für uns nicht nur fachlich sinnvoll, sondern auch strukturell hilfreich. Wenn eine Funktion nur eine klar umrissene Aufgabe hat, lässt sie sich einfacher prüfen, nachvollziehen und im Fehlerfall gezielter überarbeiten.

4.5 Nachvollziehbarkeit der Ergebnisse

Die Entscheidung, jeden Tick zu speichern und erst am Ende eine Summary zu erzeugen, war auch eine Entscheidung für Nachvollziehbarkeit. Damit bleiben Verlauf und Ursachen von Entwicklungen im Simulationslauf besser erkennbar, anstatt am Ende nur komprimierte Zahlen zu sehen. Gerade komplexe Verhaltensmuster, etwa temporäre Rückstaus an den Gates, lassen sich so auch im Nachhinein genauer analysieren.

4.6 Fairness und Realismus bei mehreren Gates

Die zusätzliche Subtick-Behandlung im Mehr-Gate-Fall haben wir vor allem deshalb aufgenommen, weil wir eine Bevorzugung einzelner Gates vermeiden wollten. Dadurch wurde die Befüllung bei mehreren Eingängen realistischer, ohne den einfacheren Ein-Gate-Fall unnötig zu belasten.

5 Organisation und Aufteilung im Team

5.1 Grundsätzliche Aufteilung

Im Team haben wir die Arbeit in drei größere Verantwortungsbereiche aufgeteilt, damit die Umsetzung parallel möglich bleibt und Zuständigkeiten klar bleiben.

5.2 Zuständigkeitsverteilung im Team

- **Simon**

- Verantwortungsbereich: Simulation und Statistik
- Zugeordnete Module und Teilbereiche:
 - * Simulation
 - * Parkhaus
 - * Demand-Berechnung
 - * gate_routing
 - * Car
 - * GenericVehicle
 - * Stats
 - * RNG

- **Luca**

- Verantwortungsbereich: Datenhandling, Struktur / Middleware und Settings-Verwaltung
- Zugeordnete Module und Teilbereiche:
 - * ConfigFileHandler
 - * SaveHandler
 - * SafetyUtils
 - * Settings
 - * StatList
 - * VehicleList
 - * types

- **Daniel**

- Verantwortungsbereich: terminalbasierte Benutzeroberfläche und Nutzerinteraktion
- Zugeordnete Module und Teilbereiche:
 - * ui

```
* ui_config
* ui_help
* ui_home
* ui_simulation
* ui_statistics
* ui_storage
```

5.3 Warum diese Aufteilung für uns sinnvoll war

Diese Aufteilung war für uns deshalb sinnvoll, weil sie entlang der tatsächlichen Systemgrenzen verlief. Die Simulationslogik, die Settings-/Speicherlogik und die UI hatten jeweils eigene fachliche Schwerpunkte und zugleich klar erkennbare Schnittstellen. Dadurch konnten wir parallel arbeiten, ohne ständig denselben Codebereich gleichzeitig verändern zu müssen.

Gleichzeitig war uns bewusst, dass diese Aufteilung nur dann funktioniert, wenn die Übergabepunkte sauber abgestimmt sind. Deshalb waren gerade Themen wie:

- Settings-Initialisierung,
- Übergabe von Statistikdaten,
- Output-Modi,
- Ownership von Datenstrukturen,
- Interface zwischen Simulation und UI

für uns keine Nebenthemen, sondern zentrale Teamthemen.

5.4 Arbeitsorganisation im Verlauf des Projekts

Die Teamarbeit war zu Beginn nicht sofort stabil eingespielt. Ein Grund dafür war, dass mehrere Teammitglieder zu Beginn akut erkrankt waren. Zusätzlich mussten wir die Toolchain rund um GitHub und Kanban zunächst selbst verstehen, aufsetzen und in eine sinnvolle Arbeitsweise überführen. Auch die aktive Kommunikation musste nicht nur eingefordert, sondern im Projektverlauf bewusst gefördert werden.

Um diese Situation zu verbessern, haben wir gezielt Besprechungstermine festgelegt. Dadurch entstanden feste und regelmäßige Zeitfenster, in denen wir offenen Abstimmungsbedarf, Schnittstellenprobleme, Entscheidungen und nächste Schritte gemeinsam besprochen haben. Rückblickend war das für uns ein wichtiger organisatorischer Schritt, weil dadurch aus losem Nebeneinanderarbeiten deutlich verlässlichere Zusammenarbeit geworden ist.

5.5 Workflow und Abstimmung

Im Arbeitsmodus spielten Branches, Reviews, Issues und Kanban als Organisationshilfe eine Rolle. Für uns war dabei weniger wichtig, ein möglichst komplexes Prozessmodell zu fahren, sondern eine Struktur zu haben, mit der wir Aufgaben sichtbar machen, Zuständigkeiten zuordnen und Probleme sauber nachverfolgen konnten.

Gerade bei Schnittstellenthemen war diese Sichtbarkeit hilfreich, weil dort Probleme selten nur einen einzigen Verantwortungsbereich betreffen. Besonders deutlich war das bei:

- Settings und Simulation,
- StatList/StatsSummary und SaveHandler,
- SaveHandler und UI,
- Parkhaus, Queue und Fahrzeuglebenszyklus.

Zusätzlich gab es im Workflow wiederholt technische Probleme mit Commits direkt aus CLion heraus. Dabei fiel insbesondere auf, dass viele Commits von Simon Ibach, unter den verwendeten Aliases `Maupher` bzw. `Toaster.-.`, trotz vorheriger Konfiguration nicht signiert wurden. Eigentlich hätte nach der Einrichtung jeder Commit regulär über Git signiert werden müssen. Warum dies in der praktischen Nutzung dennoch nicht zuverlässig funktionierte, konnte im Projektverlauf nicht abschließend geklärt werden.

Aus unserer Sicht deutet vieles darauf hin, dass die Ursache nicht nur in einer einzelnen lokalen Fehlkonfiguration lag, sondern möglicherweise in einem übergeordneten Implementierungsproblem im Zusammenspiel zwischen GitHub und der in CLion möglichen Direktverknüpfung beider Plattformen. Diese technischen Schwierigkeiten ließen sich trotz mehrfacher Anpassungen nicht vollständig auflösen und haben den Workflow zeitweise unnötig erschwert.

6 Rückblick auf die Teamarbeit & Projekt

6.1 Herausforderungen in der Zusammenarbeit

Rückblickend war unsere Teamarbeit am Anfang durchwachsen. Der Hauptgrund war nicht mangelnde Motivation, sondern die Kombination aus Krankheit, noch nicht eingespielter Toolchain und zunächst nicht ausreichend etablierter Kommunikationsroutine. Gerade bei einem Projekt, das mehrere technische Teilbereiche hat, führt schon wenig Abstimmung schnell dazu, dass jede Person in ihrer eigenen Teilwelt arbeitet.

Dazu kam, dass wir nicht nur Code schreiben mussten, sondern gleichzeitig auch ein gemeinsames Verständnis für Struktur, Begriffe und Verantwortlichkeiten entwickeln mussten. Typische schwierige Punkte waren:

- Schnittstellen zwischen Modulen,
- konsistente Benennungen rund um Tick, Zeit und Einstellungen,
- Speicherverwaltung und Ownership,
- Definition und Umfang der Statistik,
- Abgrenzung von Pflichtausgabe und optionalen Modi.

Diese Punkte waren nicht deshalb schwierig, weil sie grundsätzlich unlösbar gewesen wären, sondern weil sie mehrere Teilbereiche gleichzeitig betreffen. Genau dort war Abstimmung besonders wichtig.

6.2 Technische Herausforderungen

Auch technisch gab es mehrere Punkte, die wir im Projektverlauf konkret lösen mussten. Dazu gehörten unter anderem:

- die finale Regel gegen halbe Ticks,
- die vollständige und konsistente Initialisierung der Settings,
- die dauerhafte Verankerung von `StatsSummary` in Verbindung mit `StatList`,
- die Übergabe von geladenen Statistikdaten an die UI,
- die sichere und kompatible Entwicklung unter Windows in Verbindung mit GitHub Codespaces,
- ein Segfault im Zusammenhang mit Konfiguration und Simulation,
- ein Bug, bei dem sich das Parkhaus nur bis `MAX_SIZE - 1` füllte.

Dass solche Punkte im Kanban sichtbar bearbeitet und im Projektstand gelöst wurden, war für uns ein Zeichen, dass die Arbeitsweise mit sichtbar gemachten Problemen und konkreten Folgeschritten funktioniert hat.

6.3 Was im Team gut funktioniert hat

Trotz der anfänglichen Schwierigkeiten gibt es aus unserer Sicht mehrere Dinge, die im Projekt gut gelungen sind.

Erstens ist uns die modulare Struktur des Projekts gut gelungen. Die Trennung in Simulation, UI, Statistik, Speicherung und Konfiguration war nicht nur auf dem Papier vorhanden, sondern hat die tatsächliche Arbeit erkennbar strukturiert.

Zweitens hat sich die fachliche Aufteilung im Team grundsätzlich bewährt. Dadurch konnten wir parallel an Kernbereichen arbeiten, ohne dass alles ständig auf eine einzige Person zurückfiel.

Drittens ist es uns gelungen, die anfangs eher lockere Abstimmung schrittweise in eine verbindlichere Kommunikationsform zu überführen. Die fest eingeplanten Besprechungen haben dabei aus unserer Sicht einen echten Unterschied gemacht.

Viertens haben wir uns nicht nur auf das reine „Laufen bringen“ konzentriert, sondern die Architektur so angelegt, dass sie nachvollziehbar und erweiterbar bleibt. Dazu gehören insbesondere das Settings-Konzept, die Zwischenebene `GenericVehicle`, die Tick-Speicherung in der Statistikliste und die getrennte UI-Struktur.

6.4 Was uns leicht gefallen ist und was eher nicht

Was uns vergleichsweise gut gelungen ist, war die Zerlegung des Problems in Teilbereiche. Sobald die grobe Architektur stand, konnten wir relativ klar in Simulation, Settings/Datahandling und UI unterscheiden. Diese Teilung hat es erleichtert, Verantwortung zu übernehmen und die Arbeit zu strukturieren.

Weniger leicht gefallen ist uns die konsequente Abstimmung an den Übergängen zwischen diesen Bereichen. Dort mussten wir im Verlauf des Projekts mehrfach nachschärfen. Gerade bei modularer Arbeit waren Schnittstellen kein Nebendetail, sondern ein eigener Arbeitsaufwand. Rückblickend war das gleichzeitig eine Herausforderung und ein Lerngewinn des Projekts.

7 Fazit

Insgesamt haben wir die Aufgabenstellung als modular aufgebaute Parkhaus-Simulation in C umgesetzt. Die zentrale Idee war für uns, nicht nur eine funktionierende Simulation zu bauen, sondern eine klar gegliederte Struktur zu schaffen. Darin sind Konfiguration und Simulationslogik sauber getrennt. Auch Queue- und Gate-Verhalten bilden einen eigenen Bereich. Dasselbe gilt für das Fahrzeugmodell. Statistik, Speicherung und Benutzerinteraktion sind ebenfalls klar abgegrenzt. So entstehen in der Summe realistische Daten.

Wir haben uns bewusst für eine tickbasierte, konfigurierbare und statistisch nachvollziehbare Lösung entschieden. Zu dieser Lösung gehören die zentrale Demand-Berechnung und die anschließende Verteilung auf die Gates. Hinzu kommen FIFO-Queues pro Gate, eine durch Gate-Entry-Zeit begrenzte Einfahrt und bedarfsabhängige Subticks bei mehreren Gates. Außerdem nutzen wir die Zwischenebene `GenericVehicle` sowie die Speicherung vollständiger Tick-Daten mit anschließender Summary-Bildung.

Im Team haben wir mehrere Alternativen diskutiert und uns in den meisten Fällen für die Variante entschieden, die wartbarer, robuster, erweiterbarer und sauberer in der Zuständigkeit war. Die Zusammenarbeit war anfangs durch Krankheit, Toolchain-Einarbeitung und noch nicht etablierte Kommunikationsroutinen erschwert. Durch die konkrete Rollenaufteilung und regelmäßige Besprechungen konnten wir diese Situation jedoch deutlich verbessern.

Rückblickend sehen wir den größten Erfolg darin, dass aus mehreren technischen Teilproblemen eine zusammenhängende, modular strukturierte Projektlösung entstanden ist. Gleichzeitig hat uns das Projekt deutlich gezeigt, dass gute Teamarbeit in so einem Vorhaben nicht nur aus Aufgabenteilung besteht, sondern vor allem aus regelmäßiger Kommunikation und sauberer Abstimmung an den Schnittstellen.